IT114 Module 3 User Input Challenges

Student submission

Assignment revision: 2 | Maximum points: 10 | Student: ja692 | Submitted: 2026-10-10T02:17:52.372Z | Submission revision: 183 | Receipt: v4r_01M4HSMEFNTZ6A12YREXBF8AFY | Exported: 2026-10-10T02:17:55.288Z

Worksheet link

https://courses.ethereallab.app/assignment/v4/IT114-001-F2026/it114-module-3-user-input-challenges/ja692

Grading link

https://courses.ethereallab.app/assignment/v4/IT114-001-F2026/it114-module-3-user-input-challenges/grading/ja692

2 submission warning(s) were acknowledged.

AI disclosure: Completed; 1 entry; 1 declared AI use; 0 declared no AI use; policy v1

User Input Challenges

Complete the three starter challenges. Each challenge is worth three points; submit its source, runtime output, and explanation in separate tasks.

Command-line calculator source

Complete `CommandLineCalculator.java` using two numeric command-line arguments and the supported `+` or `-` operator.

URL prompt

Submit the GitHub URL that opens `M3/CommandLineCalculator.java` on your `M3-Homework` branch.

Requirements and potential deductions

- A nearby source comment includes your UCID, date, and a brief solution summary. (up to -0.2 points)
- The program reads two numeric operands and an operator from command-line arguments. (up to -0.3 points)
- The program accepts integer and floating-point operands. (up to -0.3 points)
- The program performs the requested addition or subtraction. (up to -0.3 points)
- The result has as many decimal places as the operand with the longer fractional part, including the stated `0.1 + 0.2` example. (up to -0.2 points)
- Invalid numeric input produces an error. (up to -0.1 points)
- An unsupported operator produces an error. (up to -0.1 points)

https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M3-Homework/M3/CommandLineCalculator.java

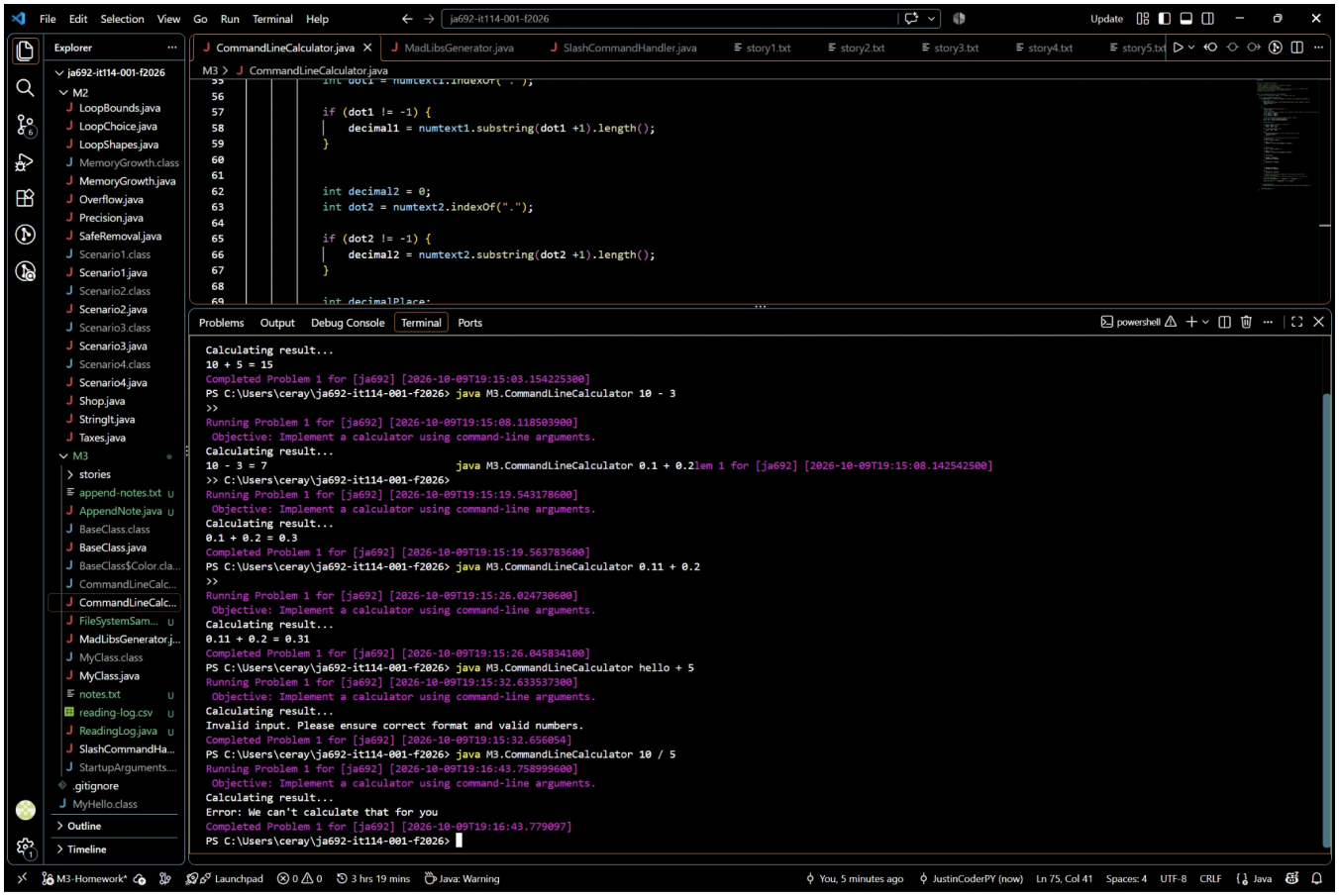
Command-line calculator output

Image evidence prompt

Upload readable terminal captures showing the complete program output for five test variations run from `M3-Homework`. Use additional images when needed to keep every run readable.

Requirements and potential deductions

- The captures show five distinct test inputs and their program output. (up to -0.6 points)
- The runs demonstrate the supported operations and both integer and floating-point input. (up to -0.2 points)
- The captures include one test with operands that have different numbers of fractional digits and show the required decimal-place result. (up to -0.2 points)



Command-line calculator explanation

Response prompt

Briefly explain how your solution reads the arguments, chooses the operation, and handles invalid input.

Requirements and potential deductions

- The response explains the solution's argument, operation, and error-handling decisions. (up to -0.5 points)

Submitted response

I used if/else statements to check whether the operator is + or - and handle the correct calculation. It also checks the number o decimal places in the inputs so the result keeps the correct format at all times, but if the user enters an invalid number or an unsupported operator, the program displays an error message instead of continuing.

Slash-command handler source

Complete `SlashCommandHandler.java` so the interactive program handles the required slash commands and invalid command input.

URL prompt

Submit the GitHub URL that opens `M3/SlashCommandHandler.java` on your `M3-Homework` branch.

Requirements and potential deductions

- A nearby source comment includes your UCID, date, and a brief solution summary. (up to -0.2 points)
- `/greet <name>` performs the intended greeting action. (up to -0.05 points)
- `/roll <num>d<sides>` performs the intended dice-roll action. (up to -0.1 points)
- `/echo <message>` performs the intended echo action. (up to -0.05 points)
- Commands are recognized without case sensitivity. (up to -0.2 points)
- `/greet` uses its required response format. (up to -0.1 points)
- `/echo` uses its required response format. (up to -0.1 points)

- `/roll` reports the dice expression and its outcome in the required format. (up to -0.2 points)
- `/quit` exits the program. (up to -0.2 points)
- An unrecognized command produces an error. (up to -0.15 points)
- An invalid format for a recognized command produces an error. (up to -0.15 points)

https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M3-Homework/M3/SlashCommandHandler.java

Slash-command handler output

Image evidence prompt

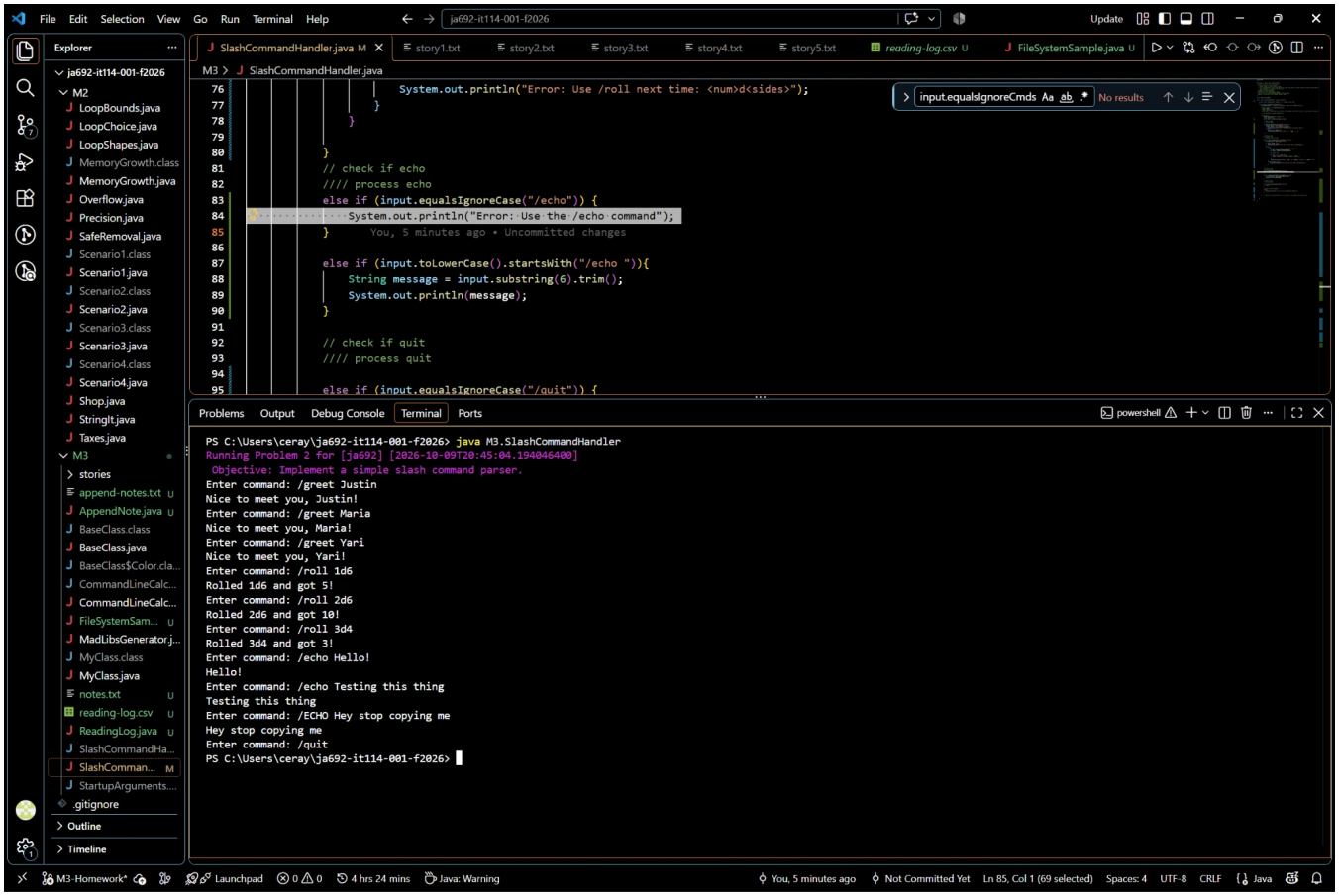
Upload readable terminal captures showing three variations each of `/greet`, `/roll`, and `/echo`, plus `/quit` ending the program. Show the output from `M3-Homework`.

Requirements and potential deductions

- The output shows three input variations for `/greet`. (up to -0.2 points)
- The output shows three input variations for `/roll`. (up to -0.2 points)
- The output shows three input variations for `/echo`. (up to -0.2 points)
- The greeting output follows its required format. (up to -0.1 points)
- The dice-roll output follows its required format. (up to -0.1 points)
- The echo output follows its required format. (up to -0.1 points)
- The output shows the program exiting after `/quit`. (up to -0.1 points)

Warning at submission: expected-items

Caption: My terminal showed all requirements needed for SlashCommandHandler tests: three /greet variations, three /roll variations, three /echo variations, and /quit successfully ending the program.



Slash-command handler explanation

Response prompt

Briefly explain how your solution recognizes commands and handles an unrecognized command or invalid format.

Requirements and potential deductions

- The response explains the command-selection and invalid-input decisions in the linked source. (up to -0.5 points)

Submitted response

Each command checks to see if the format is correct before anything is set up, and if they have put in an invalid format, like /ec without a message, it'll go to an error result. If it's not even recognized and doesn't exist in the command directory, it'll result in unrecognized command error.

Mad Libs generator source

Complete `MadLibsGenerator.java` using the five provided stories in `M3/stories`.

URL prompt

Submit the GitHub URL that opens `M3/MadLibsGenerator.java` on your `M3-Homework` branch.

Requirements and potential deductions

- A nearby source comment includes your UCID, date, and a brief solution summary. (up to -0.2 points)
- The program randomly selects one of the five provided stories. (up to -0.1 points)
- The program reads every line of the selected story into a collection. (up to -0.2 points)
- The program prompts for each placeholder. (up to -0.1 points)
- Placeholder underscores display as spaces in the prompts. (up to -0.2 points)
- The program accepts the entered word without checking whether it matches the placeholder type. (up to -0.3 points)
- Each response replaces its matching placeholder in the original line collection. (up to -0.4 points)

https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M3-Homework/M3/MadLibsGenerator.java

Mad Libs generator output

Image evidence prompt

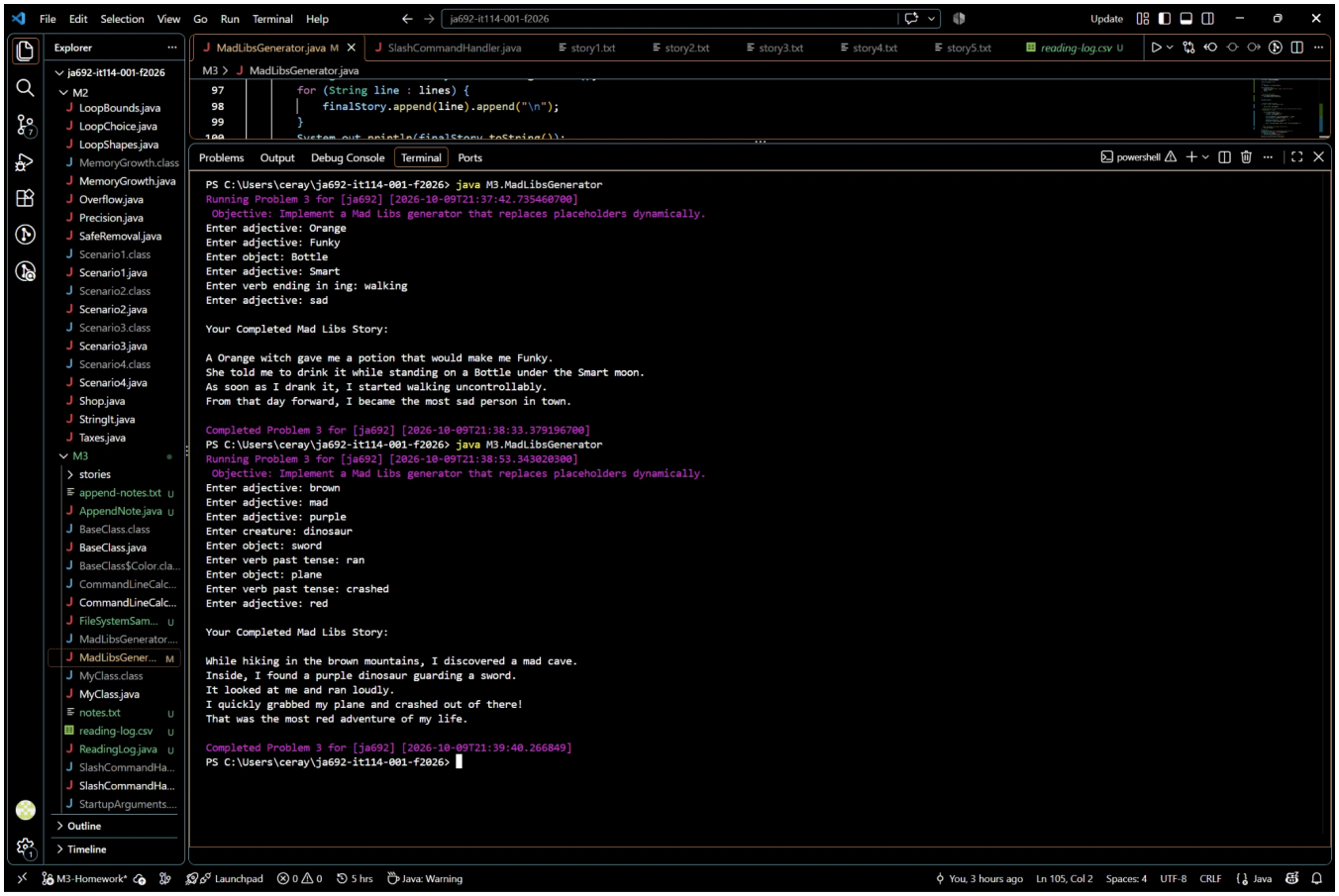
Upload readable terminal captures showing the prompts and completed output for two different stories selected from `M3/stories`, run from `M3-Homework`.

Requirements and potential deductions

- The image shows completed output for two different provided stories. (up to -0.6 points)
- The output shows prompted words substituted into their matching story locations. (up to -0.4 points)

Warning at submission: expected-items

Caption: Same thing for slash handler



Mad Libs generator explanation

Response prompt

Briefly explain how your solution selects a story, reads its lines, and replaces placeholders with responses.

Requirements and potential deductions

- The response explains story selection, line collection, and placeholder replacement in the linked source. (up to -0.5 points)

Submitted response

This code picks a random story the professor assigned to us. From there, it asks the user for every word needed to replace; it displays underscores as spaces in the prompt and replaces each placeholder with the user's input without any specific wording spelling. The updated lines are then put back into the list, and the whole story is printed out with the placeholders.

Reflection

Pull request and commit history

URL prompt

Submit the GitHub pull-request URL from `M3-Homework` into `main`.

Requirements and potential deductions

- The pull request has `M3-Homework` as its source branch and `main` as its target. (up to -0.1 points)
- Its commit history shows the untouched starter baseline, the planning commit before solutions, and subsequent solution work. (up to -0.1 points)

https://github.com/JustinCoderPY/ja692-it114-001-f2026/pull/7

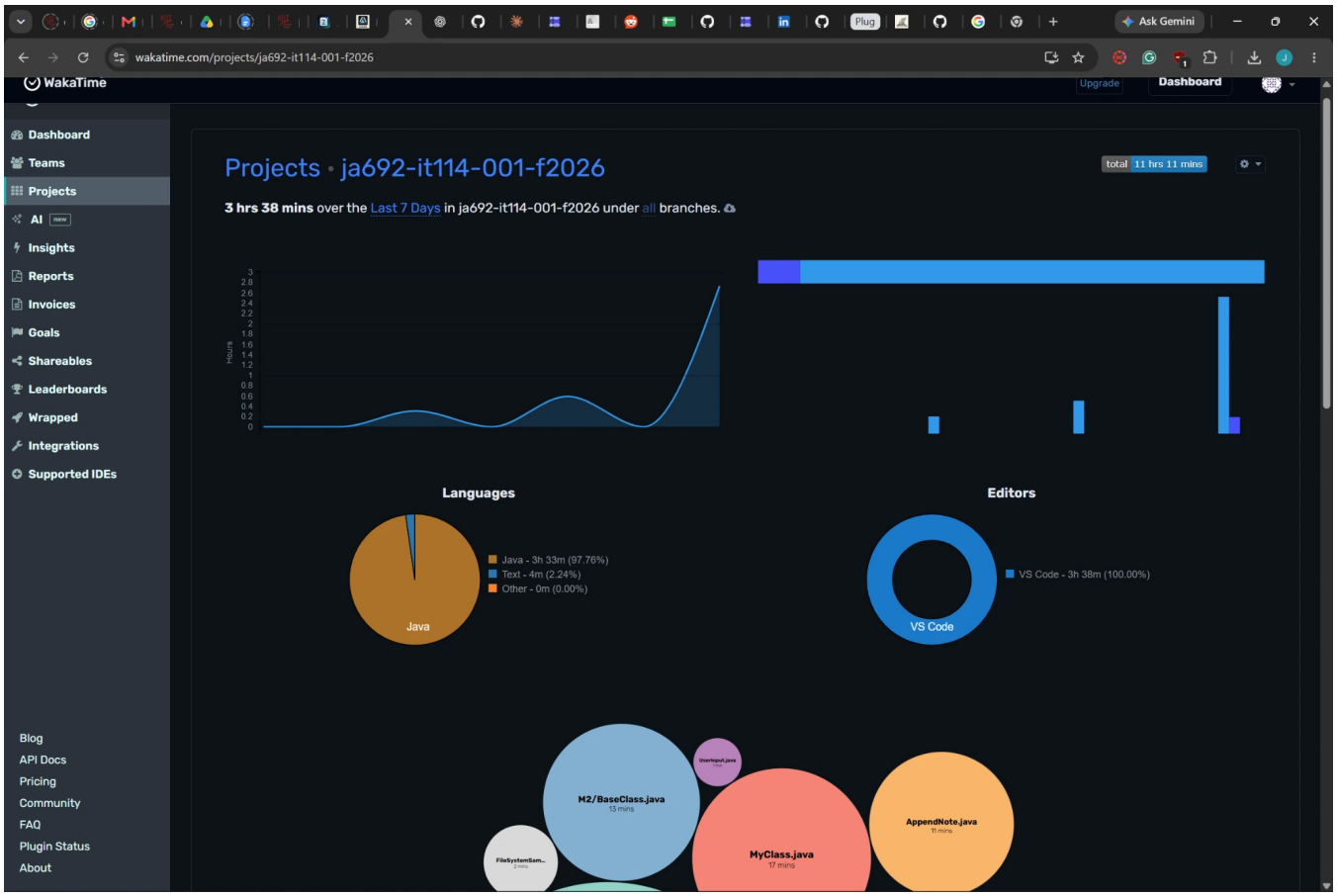
WakaTime project activity

Image evidence prompt

Upload one readable WakaTime Projects image identifying your personal IT114 repository and showing its activity during this assignment.

Requirements and potential deductions

- The image identifies your repository and shows its project activity; time duration is not graded. (up to -0.1 points)



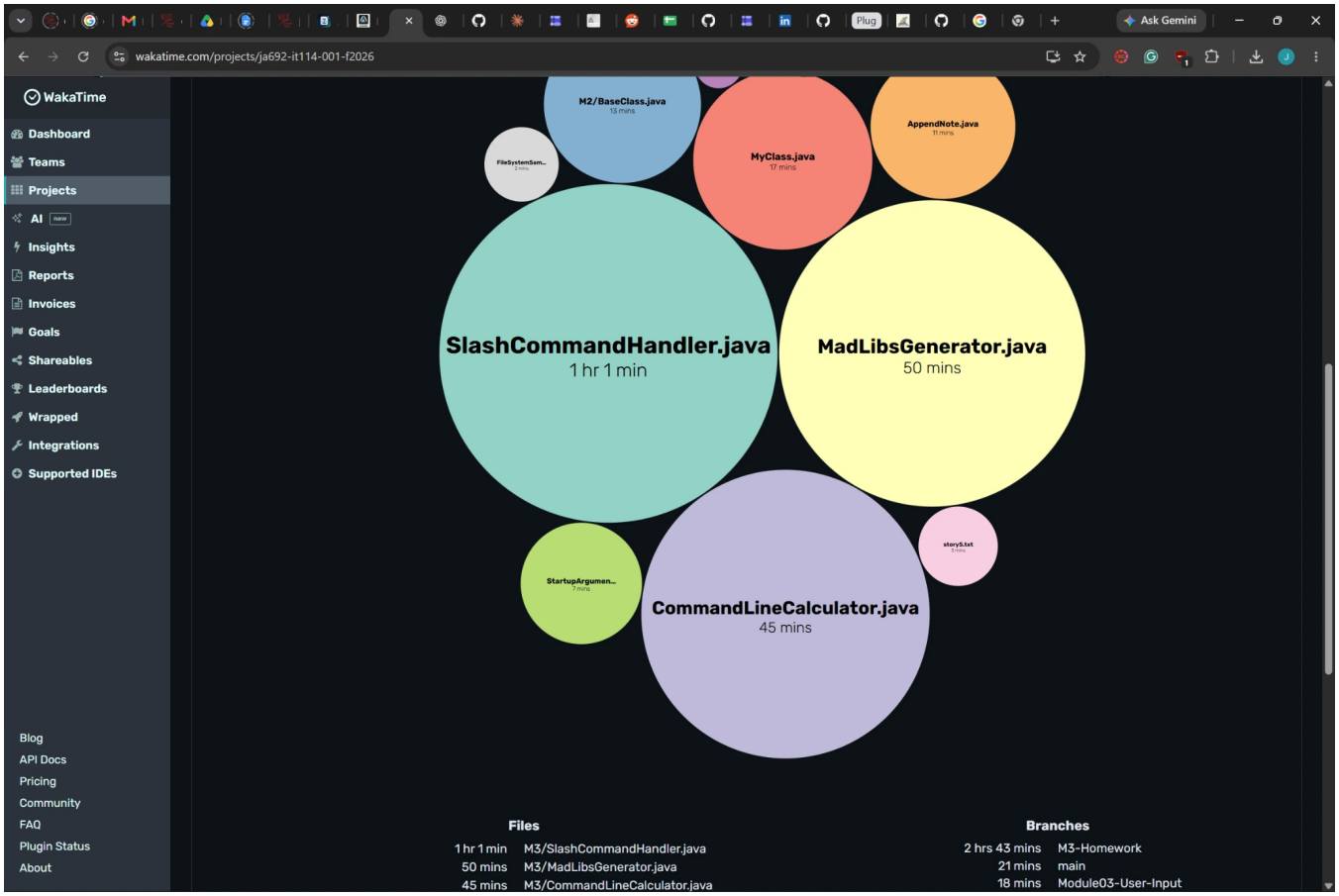
WakaTime file activity

Image evidence prompt

Upload one readable WakaTime image showing Java file activity for your personal IT114 repository during this assignment.

Requirements and potential deductions

- The image shows relevant Java file activity; time duration is not graded. (up to -0.1 points)



Learned or verified

Response prompt

What did you learn or verify while completing `M3-Homework`?

Requirements and potential deductions

- The response identifies something learned or verified through specific assignment experience. (up to -0.2 points)

Submitted response

I learned how to work with different types of user input, especially when it comes to extracting information from other files, while learning how to create simple commands. I also verified how try/catch blocks, file reading, and string replacement can be used handle errors and update the code when running it.

Easiest part

Response prompt

What was easiest or most straightforward in the `M3-Homework` work, and why?

Requirements and potential deductions

- The response identifies an easiest or most straightforward part and gives a reason. (up to -0.2 points)

Submitted response

The easiest part was understanding and creating the slash commands because they could be easily checked by using simple conditions like if/else statements, and once the input was separated correctly, each command can understood what role they s

Most difficult or uncertain part

Response prompt

What was most difficult or uncertain in the `M3-Homework` work, or still needs work?

Requirements and potential deductions

- The response identifies a concrete difficulty, uncertainty, or remaining next step. (up to -0.2 points)

Submitted response

It feels like the hardest part of the assignment was the Mad Libs generator because I basically had to be precise about reading story files and then find the right areas to do these placeholders.